

COMP8812: Data Warehousing & Big Data

Assignment 1

(iMusic Company Data Warehouse Design)



Student Name: Nikeeta Kumari

Student ID: 1582117

Course: PGDAT Semester 1 (2025)

Section A - Dimensional Modelling

a. Logical Database Design Diagram of OLAP Schema

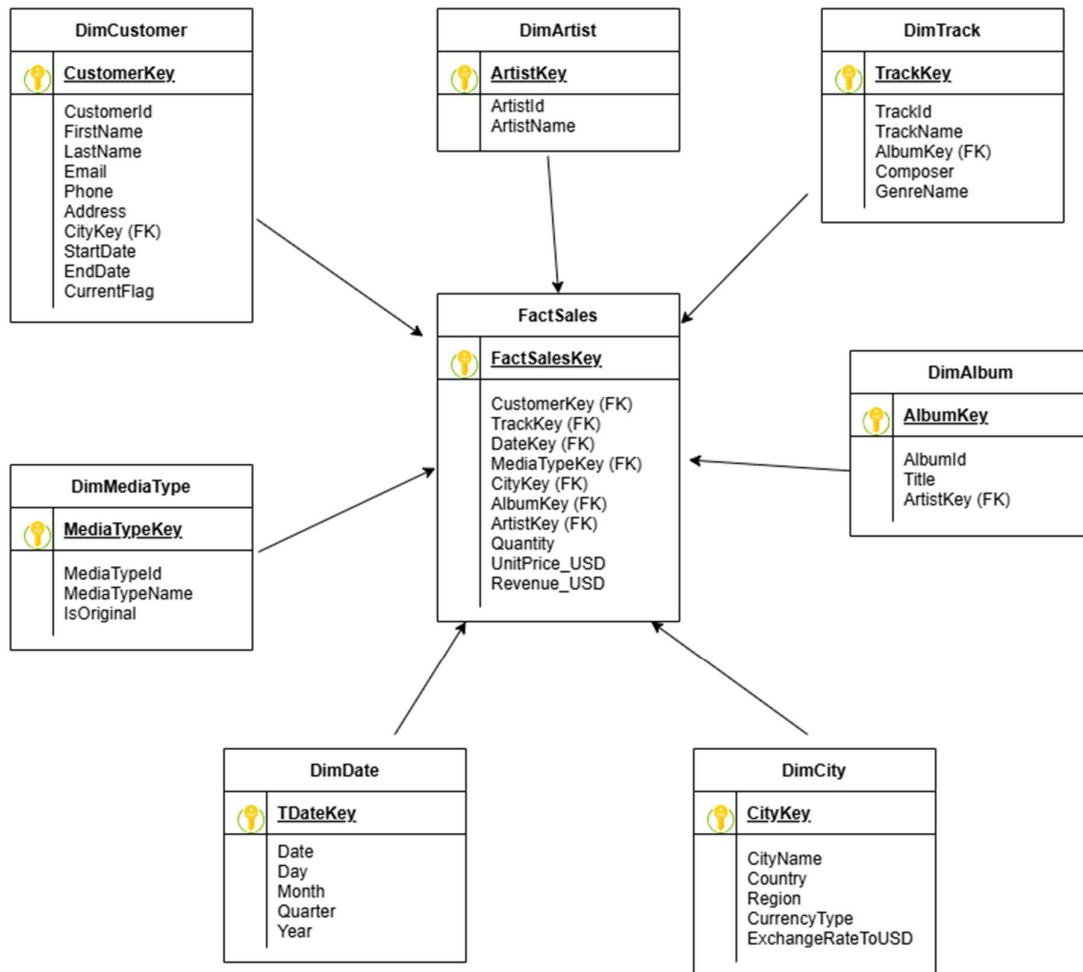


Fig: Star Schema for OLAP

I use a Star Schema with one fact table and several dimension tables. I included UnitPrice_USD and Quantity from the InvoiceLine table and put them in the fact table with a calculated column Revenue_USD. I included CurrencyType and ExchangeRateToUSD in DimCity, which is practically necessary for currency conversion requirements and for The ETL process to know what exchange rate to apply for a transaction. For the DimCity Table, I will get city names and countries from the Employee, Customer, and Invoice Table, and also exclude the Employee table as it is not directly useful in reports.

b. Schema Type and Justification

For this OLAP schema, I chose the Star Schema. It arranges data with one main fact table in the center and related dimension tables around it. This structure is easy to understand and works well for analysis. The Star Schema is a good choice because it allows fast reporting and summarizing large amounts of data. Its denormalized design means fewer joins needed through running queries, making it faster, especially for complex reports like the top-selling artists by city and year or most popular tracks by media type. It is also flexible and easy to expand. New dimension tables or data can be added without changing the whole setup. This makes it a great match for the assignment goals, like analyzing revenue, tracking customers, seeing trends over time, and converting currencies. Overall, the Star Schema is simple, fast, and easily scalable, making it the best fit for this data warehouse solution.

c. Disadvantages of Star Schema

In a Star Schema, dimension tables are purposely kept denormalized to make queries easier and run faster. But this also means some data gets repeated. For example, in a DimCity table, the same country or region might appear many times for different cities. This makes writing queries simple but uses more storage and can cause errors. If someone needs to update a country or region name, they must change it everywhere it appears. On the other hand, a Snowflake Schema avoids this repetition by breaking dimension tables into smaller related tables, like DimCity, DimCountry, and DimRegion. This keeps data cleaner and more consistent, but queries become more complex because they require more joins.

Another downside of the Star Schema is that it struggles with many-to-many relationships. It works best with one-to-many relationships, like one country having many cities. But real-world data often has many-to-many connections, like one song being in multiple playlists or one customer talking to several support agents (Employee). To handle this in a Star Schema, we need to add extra tables called bridge tables, which can make things more complicated. A Galaxy Schema (also called a Fact Constellation Schema) handles these situations better. It allows multiple fact tables and shared dimension tables, which helps when dealing with more complex data. So, while the Star Schema is great for simple reports, it may not be the best choice for more detailed or complicated analysis.

d. SCD Type for Customer Data

For this data warehouse design, I have chosen to implement an **SCD Type 2** to manage changes in the DimCustomer table. This approach is used to preserve historical information whenever there is a change in customer attributes such as city, address, or contact details. Instead of overwriting the existing record, SCD Type 2 creates a new version of the customer record with updated values while keeping the old record for historical reference. This is particularly useful for tracking a customer's movement across different regions or cities over time, which is relevant to the business's need for accurate historical reporting.

For example, suppose customerId C1 originally lived in **Berlin**. His information would be recorded in DimCustomer with a StartDate of 2023-01-01, EndDate as NULL, and CurrentFlag as TRUE. Later, if he moves to **Frankfurt**, a new record is inserted with the same CustomerId, but a new CustomerKey, the updated city name Frankfurt, a new StartDate, and CurrentFlag as TRUE. The previous record's

EndDate and current flag as FALSE are updated. This allows reports to reflect his purchases in Berlin during the earlier period and in Frankfurt after the move. The use of SCD Type 2 ensures that such changes are tracked accurately over time, which makes it the most suitable choice for the company's requirement to maintain data history and enable detailed trend analysis.

SECTION B – DATA QUALITY

a. Data Accuracy

Data accuracy refers to the degree to which data in the warehouse correctly reflects true business activities. It ensures that the stored information is correct and similar to the original data from source systems. Inaccurate data can lead to incorrect business decisions and misreporting. To maintain accuracy, it is crucial to perform validation checks during the ETL (Extract, Transform, Load) process to detect and correct any mistakes. For example, in the FactSales table, we can check if the Revenue_USD value is equal to Quantity × UnitPrice_USD. If it's not, there may be a mistake in the data entry or how the data was transformed. These records should be flagged for review to correct the errors.

b. Data Validity

Data validity ensures that the data values are in the same as predefined formats, data types, and business rules. Valid data matches the expected standards, such as proper date formats, value ranges, and data types, which is essential for meaningful analysis and reliable reporting. Invalid data can cause reports to fail or mislead users with incorrect conclusions. For example, in the DimDate table, a rule can be introduced to ensure that all entries in the Date column follow the YYYY-MM-DD format and represent actual calendar dates. Any invalid entries, such as 2023-02-30, would be detected as incorrect and not accepted or must be corrected during the ETL process.

c. Data Consistency

Data consistency refers to the uniform representation of data across the entire data warehouse. It ensures that the same information is entered in the same way in all relevant tables to avoid contradictions and confusion. Inconsistency in data can cause problems in joins between tables, show aggregation errors, and reduce the accuracy of reports. For example, we need to check how country names are stored. It is necessary that the country "United States" is stored as "USA" across all dimension tables. During the ETL process, data cleaning and standardization routines should be applied to correct any variations like "U.S." or "United States of America" to the standardized format "USA" to ensure accurate reporting and analysis.

d. Data Completeness

Data completeness means that all important fields in the data are filled in with nothing missing. If some data is missing, it can cause problems like wrong analysis, misleading results, or reports that don't work at all. It is very important in fact tables, which store numbers and keys used to connect with other tables. For example, in the FactSales table, fields like CustomerKey, TrackKey, DateKey, and Revenue_USD must always have values. They should never be empty. To make sure of this, a DQA (Data Quality Assurance) rule can be used to check that all these important fields are filled in

for every record. If any of them are missing, that record should be flagged, rejected, or sent for correction so that the data used in reports is complete and reliable.

e. Uniformity of Data

Uniformity of data means that the same units, formats, and scales are used throughout the warehouse to ensure compatibility and comparability. This is especially important in environments where data is sourced from multiple regions that may use different currencies and units of measure. Lack of uniformity can lead to invalid comparisons. For the iMusic data warehouse OLAP design, a DQA rule must be set that all financial values, such as UnitPrice and Revenue, are converted and stored in a single currency (USD). This conversion is performed during ETL using the **ExchangeRateToUSD** column from DimCity. Ensuring this uniformity allows accurate aggregation and comparison of sales data across countries and regions in a consolidated, meaningful way.

Conclusion

The OLAP star schema designed for iMusic provides a simple, high-performance framework for analyzing global music sales data. It supports efficient reporting through a central fact table linked to clear dimension tables while also incorporating **SCD Type 2** in DimCustomer to track historical changes. Currency conversion is managed via DimCity, ensuring all financial data is standardized in USD. The design integrates key **data quality assurance rules** to maintain accuracy, validity, consistency, completeness, and uniformity. The solution meets the business's analytical needs and is scalable for future reporting requirements.